

Practical # 13

Objective:

Construct and implement a Binary Search Tree (BST) with insertion, search, and deletion operations.

Theory:

A tree is a very popular non-linear data structure used in a wide range of applications. Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition. A tree data structure can also be defined a collection of data (Node) which is organized in hierarchical structure recursively. A tree in which every **node can have a maximum of two children** is called **Binary Tree**. A binary tree data structure is represented using two methods:

- ✓ **Array Representation**
- ✓ **Linked List Representation**

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal. There are three types of binary tree traversals.

1. **In - Order Traversal (leftChild - root - rightChild)**
2. **Pre - Order Traversal (root - leftChild - rightChild)**
3. **Post - Order Traversal (leftChild - rightChild - root)**

1. In - Order Traversal (leftChild - root - rightChild)

Following operations are carried out to traverse a binary tree using In-Order way:

- a) Traverse the left most sub-tree starting at the left external node
- b) Visit the root
- c) Traverse the right sub-tree starting at the left external node

1. Create an empty Stack S
2. Initialize current node as root
3. Push the current node to S and set Current= Current Left **until** Current = Null
4. If Current is Null and Stack is not empty then:
 - a. POP the item from S
 - b. Print the Popped item, Set Current Right
 - c. Go to Step 3
5. If Current is Null and Stack is empty then Exit

2. Pre - Order Traversal (root - leftChild - rightChild)

Following operations are carried out to traverse a binary tree using Pre-Order way:

- a) Visit the root
- b) Traverse the left most sub-tree starting at the left external node
- c) Traverse the right sub-tree starting at the left external node

1. Create an empty Stack S
2. Push the root on Stack
3. While the Stack is not empty
 - a. POP the item from Stack and print
 - b. PUSH its children in the Stack

3. Post - Order Traversal (leftChild - rightChild - root)

Following operations are carried out to traverse a binary tree using Post-Order way:

- a) Traverse the left most sub-tree starting at the left external node
- b) Traverse the right sub-tree starting at the left external node
- c) Visit the root

1. Create an empty Stack S
2. Do following While root is not NULL
 - a. PUSH root's right child and then root in stack
 - b. Set root as root's left child
3. POP an item from stack and set it as root
 - a. If the POPPED item has a right child and the right child is at top of stack, then remove the right child from the stack, PUSH the root back and set root as root's right child
 - b. Else print root's data and set root as NULL
4. Repeat step 2 and 3 while stack is not empty

In this lab, students will learn how to:

- **Construct** a Binary Search Tree (BST).
- **Implement** insertion, search, and deletion operations in BST.
- **Apply** traversal techniques (inorder, preorder, postorder).
- **Analyze** the performance of BST operations in best, average, and worst cases.
- **Evaluate** the efficiency of BST compared to linear data structures.

C++ program: Write C++ program to traverse binary tree using In-order traversal method.

```
#include <iostream>
using namespace std;
/* A binary tree node has data, pointer to left child
and a pointer to right child */
struct Node {
    int data;
    struct Node *left, *right;
    Node(int data)
    {
        this->data = data;
        left = right = NULL;
    }
};

/* Given a binary tree, print its nodes in inorder*/
void printInorder(struct Node* node)
{
    if (node == NULL)
        return;

    /* first recur on left child */
    printInorder(node->left);

    /* then print the data of node */
    cout << node->data << " ";

    /* now recur on right child */
    printInorder(node->right);
}

/* Main program */
int main()
{
    struct Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);
    return 0;
}
```

OUTPUT

```
Inorder traversal of binary tree is
4 2 5 1 3
```

Review Questions/ Exercise:

1. Construct a C++ program to **insert nodes** into a Binary Search Tree.
2. Implement a program to **search for an element** in a BST and display whether it is found or not.
3. Develop a program to **delete a leaf node** from a BST.
4. Extend the deletion program to **delete a node with one child**.
5. Extend the deletion program to **delete a node with two children** (using inorder successor/predecessor).
6. Implement and display BST traversals: **inorder, preorder, postorder**.
7. Write a program to **count the number of nodes** in a BST and determine the tree height.
8. Develop a program to **find the minimum and maximum value** in a BST.

Name: _____

Roll #: _____

Date: _____

Subject Teacher